PROJECT GUIDE · ARCHITECTURE · FEATURES · OPERATIONS

Bisectr

Measure performance regressions before they merge. Find the likely culprit when they slip through.

> **In one sentence:** Bisectr compares container images under the same repeatable HTTP workload on AWS Fargate, evaluates latency, errors, and estimated resource cost, and reports a gate verdict with evidence.

Technical overview, user workflows, architecture, setup, safety model, validation evidence, and product direction.

Repository: aryan-95/bisectr · Guide prepared 30 October 2026 · Project status: prototype / hackathon build

This guide describes the checked-in implementation and observed artifacts. Optional or unconfigured infrastructure is identified as such.

1. Executive overview

Software changes can make a service slower, less reliable, or more expensive while still passing functional tests. Bisectr adds a performance check to the development workflow: it runs a baseline image and a candidate image side by side, sends both the same configured workload, measures their behavior, and produces a PASS, WARN, or FAIL verdict.

If a regression is discovered after several commits, Bisectr can test intermediate images to narrow down the first bad revision. A separate fix-verification command can check whether a proposed fix restores performance relative to a known-good image.

For	Teams maintaining containerized HTTP services that want a repeatable, automated performance regression signal during code review.
Core value	Relative A/B evidence tied to immutable image/commit references, rather than a subjective “it feels slower” report.
Execution	Python CLI and GitHub/GitLab automation orchestrate short-lived AWS ECS Fargate probe tasks.

Current status	Working prototype with tests and AWS-backed capabilities; requires the user's AWS/GitHub configuration and service-specific validation.

> **Scope:** Bisectr is a measurement and investigation tool, not a deployment platform. The current alarm automation is opt-in, not applied/live by default, and tracks successful main image builds—not confirmed production deployments.

Problems it addresses

- Slowdowns that functional tests do not catch.
- Performance regressions hidden by aggregate metrics or inconsistent test runs.
- Time-consuming manual searches through a commit history after a regression appears.
- Unclear operational impact: route-level latency/error evidence and an approximate cost model put changes into context.

2. Architecture at a glance

The trusted Bisectr CLI coordinates the comparison. The application and load runner execute together in a short-lived Fargate task; the runner sends requests to the app over localhost and publishes a structured result to CloudWatch Logs.

Developer opens PR → GitHub Actions builds candidate image → Trusted Bisectr CLI selects baseline + candidate

↓

ECR stores images ← ECS starts matching Fargate probe tasks → Baseline and candidate run concurrently

↓

CloudWatch Logs returns runner measurements → Statistical judge compares results → PR check/comment + HTML report + history

Key components

Component	Responsibility
Bisectr CLI / Python engine	Configuration, image checks, probe orchestration, judgement, bisect, report generation, history, and cost estimation.

Demo app / customer image	The HTTP service under test. The demo app has injectable latency, CPU, payload-size, and error-rate regressions.
Runner container	Open-loop load generation, warm-up, route-level measurements, and structured result logging.
AWS ECR + ECS/Fargate	Image storage and isolated, temporary probe-task execution.
CloudWatch Logs	Transport for probe results back to the CLI; the CLI waits for tasks and reads their runner stream.
GitHub Actions / GitLab CI	Build images, invoke trusted Bisectr code, and report a CI verdict. GitHub CI uses OIDC for AWS access.
DynamoDB + S3 (optional configured storage)	TTL-bound baseline cache/run records and private HTML report objects. Local mode writes under project-local cache and runs directories.
SSM, Budgets, reaper	Kill switch, budget controls/alerts, and cleanup safeguards for old Fargate tasks and task-definition revisions.

Probe task security boundary

The probe task contains the app and runner containers in the same Fargate task. They communicate on localhost. Terraform configures no inbound security-group access and permits outbound HTTPS for image pulls/log shipping. The task has an execution role for infrastructure operations but deliberately has no application task role. The PR candidate image is built in a job without AWS credentials; a separate trusted job handles AWS operations.

This is defense in depth, not a claim that arbitrary application code is safe. Use disposable environments/data and review IAM, network, and CI settings before production use.

3. What Bisectr measures and decides

Paired performance gate

1. Check that baseline and candidate images exist in ECR.
2. Start the same configured workload against both images at approximately the same time.
3. Collect latency samples, request/error counts, CPU-seconds per request, memory, and endpoint-level statistics.
4. Evaluate statistical significance, practical effect size, replica agreement, error change, and configured budgets.

5. Return PASS, WARN (insufficient/inconclusive evidence), or FAIL. When evidence is inconclusive, the gate can repeat with a longer run.

The default latency rule combines a one-sided Mann–Whitney comparison, a minimum p95 ratio, a minimum absolute p95 difference, and agreement across replicas. Error-rate increases have a separate threshold. Exact thresholds are configurable; review the actual `bisectr.yaml` used for every reported result.

Route budgets and multiple workloads

Workloads are configured as weighted HTTP endpoints with methods, paths, optional JSON bodies, request rate, duration, and warm-up. Multiple named profiles can represent normal and peak traffic; `bisectr suite` runs the configured profiles and fails the suite if any profile fails. Per-endpoint `max_p95_ratio` and `max_error_delta` budgets expose route regressions that aggregate measurements could hide. Under-sampled endpoints are reported as insufficient evidence—not silently treated as good.

Cost estimate

Bisectr uses measured CPU-seconds per request and target RPS/utilization to estimate task counts and a monthly Fargate cost delta when regional vCPU and memory prices are configured. This is a sizing estimate, not the AWS invoice; it omits other billable resources and depends on representative workload and pricing assumptions.

Commit bisect and fix verification

Bisect searches between known-good and known-bad image revisions, probes selected intermediate commits, handles missing images as skips, and verifies the suspected culprit against its parent. It can return a found culprit, ambiguous window, flaky result, or not-reproducible result. A cautious ambiguous result is preferable to a false certainty. `verify-fix` compares a proposed fixed image with a known-good baseline and reports resolution only when the measurement passes.

4. Features and available workflows

Feature	How to use it	Outcome
Single gate	`bisectr gate --candidate SHA --baseline SHA`	Side-by-side measurement, verdict, report, optional PR comment/check.
Workload suite	`bisectr suite --candidate SHA --baseline SHA`	Runs multiple configured workload profiles; any failing profile fails the suite.
Commit bisect	`bisectr bisect --good SHA --bad SHA`	Narrows the first bad commit; optional issue reporting.

Fix verification	`bisectr verify-fix --baseline GOOD --candidate FIX`	Checks whether the candidate restores performance against the known-good image.
Repeatable validation	`bisectr validate --baseline IMAGE --candidate IMAGE --trials N`	Runs uncached repeat trials and writes CSV evidence.
Evaluation harness	`bisectr eval --image SHA --trials N`	Runs controlled injected regressions and bisect trials; consumes AWS capacity and time.
Noise simulation	`python eval/noise_sensitivity.py`	Offline model of bisect outcomes as verdict flips increase; simulation, not AWS evidence.
History and dashboard	`bisectr trends`, `bisectr dashboard`	Summaries and local HTML visualization of recorded run history; does not launch probes.
CI checks	GitHub Actions or protected-branch GitLab CI	Automates image build, measurement, and reporting in a developer workflow.
Alarm-triggered investigation (optional)	Terraform opt-in plus a matching CloudWatch alarm	Bisects the latest two successful main image builds and may open a verified, unmerged revert PR.

GitHub PR flow

The candidate image is built without AWS credentials and uploaded as an artifact. A trusted-base job downloads it, assumes the configured AWS role through OIDC, publishes the image, executes the gate, and posts the result. The workflow restricts cloud-backed PR runs to eligible same-repository contributors; it intentionally does not run the privileged gate on fork PRs. Require the named check through repository branch rules only after the check has successfully reported.

GitLab flow

The documented GitLab pipeline runs ordinary tests and limits AWS-backed gate execution to a protected default-branch context using a project/ref-scoped AWS trust relationship. The current described setup is not a general merge-request gate; confirm its pipeline behavior and runner prerequisites before presenting it as merge blocking.

5. Alarm workflow and operational safety

The optional alarm path is designed as a reviewable recovery aid, not autonomous production rollback:

1. A successful main image build publishes the app and runner images, then records the previous and current successful main build SHAs in SSM Parameter Store.
2. EventBridge matches CloudWatch alarm transitions into ALARM with a configured name prefix.
3. A Lambda validates that SHA range, deduplicates event retries with DynamoDB, retrieves a scoped GitHub token from Secrets Manager, and dispatches the alarm workflow.
4. The workflow bisects the range. If a culprit is verified, it creates a separate revert branch, builds the reverted demo-app image, and runs fix verification.
5. It opens a revert PR only after verification passes. If the culprit or fix is not verified, it opens an investigation issue instead. It never auto-merges.

> **Critical limitation:** this project does not deploy the demo application. The SHAs represent successful main image builds, not the version running in production. A production alarm may refer to another version. Do not call this production rollback or claim that a passing synthetic check proves production recovery.

Operational controls

- SSM kill switch at `/bisectr/enabled` can block new probe runs.
- Maximum probe/task concurrency and probe timeouts limit resource use; Ctrl+C/error cleanup stops active tasks.
- A reaper stops stale tasks and deregisters old task-definition revisions.
- AWS Budgets and SNS notifications provide spend visibility; configuration may block new runs at the configured budget threshold.
- Cloud resources have ongoing costs. Check the target AWS account, region, budget, active tasks, and image tags before starting evaluation.

Enabling alarm automation requires

Set `enable_alarm_bisect=true`; provide the repository, alarm prefix, and Secrets Manager ARN for a fine-grained GitHub token; allow Actions to create PRs; apply Terraform; publish at least two successful main image builds; and test a controlled alarm in non-production. The feature is disabled by default and was not enabled/applied as of this guide.

6. Setup and practical command guide

Local development

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
python -m pytest -q
bisectr --help
```

Python 3.12 or newer is required. The development extras include pytest and Hypothesis. Local tests do not require Fargate and are the first validation step.

AWS-backed setup (operator work)

Provision the development stack using Terraform 1.10+ and AWS credentials with the required permissions. Before Terraform initialization, create a separate state bucket with encryption, versioning, and public access blocked. Apply Terraform, configure GitHub OIDC/Actions permissions and repository secrets, generate/review `bisectr.yaml`, set pricing and safe test workloads, and push the runner/app images. Never overwrite an existing customized config without a backup.

```
aws sts get-caller-identity
terraform -chdir=infra/envs/dev output
python scripts/gen_config.py
./scripts/push_images.sh
```

Confirm the AWS account/region, ECR image tags, kill switch, Fargate capacity/budget, security group, subnets, log group, and runner image before running probes. Image tags must be present in ECR; placeholder strings are not valid tags.

Short demonstration

```
make gate-ok       # A/A control: same image on both sides; expect PASS
make gate-bad      # injected slow_dependency regression; expect FAIL
bisectr trends --kind gate
bisectr dashboard --out runs/dashboard.html
```

The gate commands launch Fargate tasks and may incur AWS charges. A failing exit code for `make gate-bad` is expected because the demonstration deliberately introduces a regression. The trends/dashboard commands only read saved history.

Repeat trials and evaluation

```
bisectr validate --config bisectr.yaml \
  --baseline BASELINE_IMAGE_TAG --candidate CANDIDATE_IMAGE_TAG \
  --trials 5 --out eval/service-validation.csv

bisectr eval --config bisectr.yaml \
  --image "$(git rev-parse HEAD)" --trials 20 --bisect-trials 3 \
```

```
  --seed 20261029 --out eval/judge-results.csv
python eval/report.py eval/judge-results.csv
```

Use your actual immutable image tags. Evaluation can take a long time and launch many tasks; do not run it alongside another Fargate test. Resume an interrupted run only when reusing the exact image, configuration, scenario file, seed, trial counts, and output CSV.

7. Use cases and product fit

Use case	Example	What Bisectr contributes
PR performance review	New endpoint implementation or dependency update	Automated baseline/candidate comparison, visible check and measurement summary before merge.
Endpoint SLO/budget monitoring during development	Checkout p95 is more important than a low-traffic admin route	Weighted traffic mix and per-route p95/error budgets.
Regression investigation	Latency increase observed after several main builds	Image/commit bisect, verification, and evidence-rich issue/report.
Cost-impact screening	CPU work increases per request at a target throughput	Estimated task capacity and monthly cost delta under explicit pricing assumptions.
Release validation	Compare two immutable images using staging-safe data	Repeatable uncached A/B trials and CSV results.

Potential users and adoption

Likely early users are platform engineering, SRE, backend teams, and maintainers of containerized APIs on AWS who can provide a safe representative workload and reproducible images. A sensible adoption path is: start with a local A/A control, run against a staging service, establish false-positive behavior, add a non-blocking PR check, then require the check only after trust and operational readiness are demonstrated.

For a startup/product direction, position Bisectr as a developer-facing performance regression gate rather than a generic monitoring replacement. A production offering would still need tenant isolation, customer-owned cloud roles, secret management, pricing/cost controls, durable audit trails, broader runtime support, deployment-aware release metadata, and a clear support/SLO model. These are product requirements—not claims that the prototype already provides them.

What it is not

- Not an APM, continuous production monitoring platform, or autoscaler.
- Not a production traffic replay system or a substitute for privacy/data safety review.
- Not an automated deployment or direct rollback mechanism.
- Not proof that an arbitrary service will have the same results as the demo app.

8. Validation evidence and interpretation

Results depend on the exact image, scenario, region, task size, price inputs, sample count, and time. The table below describes the artifacts checked for this guide; it is not a universal accuracy claim.

Evidence	Observed result	How to interpret it
Automated unit tests	80 tests passed in the last recorded validation run.	Good evidence for implementation logic, not proof of AWS deployment correctness.
Current AWS evaluation CSV	40 gate trials across n_plus_one and slow_dependency (20 each); four bisect trials recorded (three n_plus_one, one slow_dependency).	Partial capture. It does not contain all requested scenarios or noise-only controls. Do not publish as a completed 20-trial evaluation.
Current partial evaluation summary	40/40 regression gates detected; false-positive rate is unavailable; four of four recorded bisect trials found a culprit; no wrong culprit in those four.	Promising but small/partial sample. A missing control means there is no measured false-positive estimate here. Four bisects are too few to establish robust accuracy.
Controlled go-httpbin comparison	Three trials of an intentionally delayed image vs. a baseline: all three FAIL; p95 ratios approximately 8.97×, 12.99×, and 9.22×.	Controlled injected-delay demonstration on a public open-source service image, not evidence that Bisectr found a naturally occurring regression or validates production traffic.
Noise sensitivity	Offline simulation models bisect outcomes under configured verdict-flip rates.	Simulation only. It is not AWS data or a substitute for repeated control and regression trials.

Limitations that affect conclusions

- Workloads must represent the target service; a mismatch can create false confidence or noise.
- Probe runner shares Fargate task CPU with the app, so results are relative measurements rather than production latency.

- Bisect works best for reproducible, roughly monotonic regressions. Reverts, intermittent behavior, or noisy verdicts can yield ambiguous/flaky results.
- Small trial counts do not establish reliable false-positive/wrong-culprit rates. Report sample sizes and uncertainty.
- Cost model is a capacity estimate using user-configured prices, not an AWS bill.
- Write endpoints require disposable test data. Current probe configuration does not inject credentials or anonymize production data.
- The existing alarm SHA tracking is build-aware, not deployment-aware.

9. Repository map and next improvements

Path	Contents
`src/bisectr/`	CLI, config, probe orchestration, gate judgement, bisect, reporting, storage, guardrails, and cost model.
`runner/`	Containerized load generator and measurement emitter.
`demo-app/`	HTTP demo service and controllable regression modes.
`infra/envs/dev/`	Terraform for ECR, ECS/Fargate, IAM/OIDC, logs, storage, budget, cleanup, and optional alarm dispatch.
`.github/workflows/`, `.gitlab-ci.yml`	Build, gate, bisect, and CI integration workflows.
`lambdas/`	Task reaper and optional CloudWatch-alarm dispatch handler.
`eval/`, `tests/`	Scenario evaluation, reports, noise simulations, and automated tests.
`docs/`	Demo, validation, alarm-automation, runbook, and project guide documentation.

Highest-value next steps

1. Finish a complete, reproducible evaluation with all scenarios, sufficient repeated bisects, and a noise-only A/A control; publish the exact CSV, config, image SHA, region, and pricing assumptions.
2. Run more repeated control and known-regression trials to estimate false-positive and detection rates with meaningful uncertainty.
3. Validate a second service using safe read-only routes and realistic staging workload; document methodology and limitations.

4. Establish a working, repeatable GitHub PR demo, confirm a successful baseline image and CI secret/OIDC configuration, and test a required check in a dedicated repository/ruleset.
5. For production relevance, integrate actual deployment SHAs only after healthy deployment, map alarms to services/releases, and verify recovery with post-deploy health checks. Keep human approval for reverts.
6. For a SaaS path, design customer isolation, cloud access onboarding, tenancy, auditability, data retention, and transparent billing before accepting customer workloads.

> **Takeaway:** Bisectr is a measurable performance quality gate with an investigation workflow. Its strongest demonstration is a controlled A/A pass followed by a known injected regression fail, with the measurements and safety boundaries shown openly.

Supporting material: README, docs/demo.md, docs/validation.md, docs/alarm-automation.md, docs/runbook.md, eval/results-20-trials.csv, eval/go-httpbin-case-study.csv. Numbers and activation status should be refreshed before external publication.